

Como Ensinar Inspeção Visual Automática de Qualidade com Edge AI – Francisco

Plano de Aula · Laboratório presencial · Projeto 2 da Matriz Edge AI / Visão Computacional

PÚBLICO-ALVO

Estudantes de nível técnico ou superior com Python básico

DURAÇÃO

3 horas (180 min) em laboratório

FORMATO

Duplas, projeto real, laboratório guiado

PROJETO

Inspeção de latas em esteira: conforme × não conforme

TECNOLOGIAS

YOLO26n · NCNN · FastAPI · Docker (amd64/arm64)

HARDWARE ALVO

Raspberry Pi 5 (demonstração opcional)

Problema que guia a aula: numa linha de envase, latas amassadas, sem rótulo ou sem cor passam a 2 latas por segundo e chegam ao cliente. Inspeção humana cansa e falha. **Como um computador do tamanho de um cartão de crédito, sem internet e sem placa de vídeo, pode decidir em tempo real quais latas tirar da linha?**

1. Objetivos de aprendizagem (Taxonomia de Bloom revisada)

Ao final da aula, cada estudante será capaz de:

Nível	Objetivo (verbo observável)	Evidência na aula
Lembrar	Identificar os elementos de uma detecção: caixa, classe e confiança.	Quiz relâmpago 1
Entender	Explicar por que processar na borda (Edge AI) em vez de enviar imagens para a nuvem nesse cenário.	Explicação da dupla ao professor
Aplicar	Executar o sistema em um container Docker e usar os endpoints HTTP de detecção (JSON e imagem anotada).	Respostas da API salvas pela dupla
Analisar	Comparar a latência do mesmo modelo em PyTorch e em NCNN na CPU e relacionar com a velocidade da esteira.	Tabela de benchmark da dupla
Avaliar	Decidir o limiar de confiança e justificar se o sistema atende a linha em tempo real na Raspberry Pi 5.	Ficha de decisão técnica
Criar	Propor uma melhoria (nova classe, alerta MQTT, quantização) para a próxima versão do sistema.	Desafio extra / próximos passos

2. Metodologia ativa e justificativa

Estratégias combinadas

- Aprendizagem Baseada em Projetos (PBL):** a aula inteira gira em torno de um problema industrial real com entregável concreto: uma decisão técnica sobre colocar ou não o sistema na linha.
- Programação em pares (piloto e navegador):** o piloto digita e o navegador lê a instrução e confere o resultado. Os papéis trocam a cada bloco de 20–25 min.

- **Laboratório guiado em degraus:** cada degrau tem um "checkpoint" visível (container no ar, primeira resposta JSON, primeira imagem anotada, tabela de latência). Ninguém avança no escuro.
- **Instrução por pares (Peer Instruction):** nos quizzes, cada estudante vota sozinho, discute com a dupla e vota de novo antes da explicação.

Por que não uma aula só expositiva

- Uma meta-análise de 225 estudos em ciências e engenharia (Freeman et al., 2014) mostrou que estudantes em aulas expositivas tradicionais tiveram **1,5 vez mais chance de reprovação** do que em aulas com aprendizagem ativa.
- O público tem só Python básico. Fórmulas de redes neurais não o ajudariam, mas ver uma lata amassada ser marcada em vermelho cria o significado que sustenta os conceitos.
- Turmas heterogêneas: em duplas, quem tem mais prática em terminal apoia quem tem menos. O desafio extra ocupa quem termina antes, sem atrasar o grupo.
- Os erros reais do laboratório (build lento, modelo não carrega, latência alta) viram conteúdo da aula, não interrupção.

3. Requisitos técnicos e pedagógicos

Infraestrutura (por dupla)

Item	Especificação
Computador do laboratório	Linux ou Windows com Docker (Desktop ou Engine), 8 GB de RAM, 15 GB livres, navegador e terminal
Imagem Docker	<code>inspecao-latas</code> já montada em cada máquina antes da aula (a montagem leva vários minutos e baixa cerca de 1 GB)
Repositório do projeto	Clonado na máquina: código, modelo NCNN, vídeo da esteira e pasta de imagens de teste
Imagens de teste	20 quadros da esteira com latas boas e com defeito (pasta <code>dataset/imagens</code>)
Rede	Rede local; internet não é necessária durante a aula (o sistema roda offline)

Infraestrutura (sala)

- Projetor e computador do professor com a mesma imagem e o vídeo da esteira.
- **Opcional:** 1 kit Raspberry Pi 5 (fonte, SSD 32 GB, câmera USB, módulo relé + LED simulando o soprador) para demonstração ao vivo. A aula funciona sem ele.

Conhecimento prévio necessário

Precisa saber	Não precisa saber
Python básico: variáveis, listas, dicionários, funções; abrir um terminal e rodar um comando	Matemática de redes neurais, treino de modelos, Docker avançado, programação embarcada

Nivelamento: uma folha de comandos (Docker, curl, navegador) é entregue impressa. Para quem nunca usou terminal, o Swagger (`/docs`) permite fazer tudo pelo navegador.

4. Sequência didática (cronograma)

Tempo	Etapa	O que acontece	Metodologia / checkpoint
0–10 min	Gancho	Vídeo da esteira com latas defeituosas passando. Pergunta: "quantas latas ruins vocês viram?" Revelação do número real. Por que um inspetor humano falha.	Problema real (PBL)
10–15	Objetivos	"Ao final da aula, vocês vão colocar um detector de defeitos para rodar e decidir se ele aguenta a linha." Formação das duplas e dos papéis.	Contrato da aula
15–35	Conceitos essenciais	Classificar × detectar × segmentar · caixa, classe e confiança · modelo pré-treinado e ajuste fino · Edge AI: por que não a nuvem (latência, rede, privacidade, custo).	Expositiva curta + Quiz relâmpago 1 (Peer Instruction)
35–55	Degrau 1: subir o sistema	<code>docker compose up -d</code> , conferir <code>/saude</code> , abrir o Swagger. Professor mostra o Dockerfile: o que é imagem, container, porta, <code>restart: always</code> .	Checkpoint: API no ar
55–80	Degrau 2: detectar	Enviar uma imagem ao <code>/detectar</code> (JSON) e ao <code>/detectar/imagem</code> (PNG). Ler o JSON: classe, confiança, caixa, conformes × não conformes, tempo em ms.	Checkpoint: 1ª imagem anotada · troca de papéis
80–90	Intervalo	—	—
90–115	Degrau 3: desempenho	Rodar o benchmark PyTorch × NCNN na CPU. Comparar com a referência oficial da Raspberry Pi 5 (NCNN ≈ 67 ms por imagem). Conta da dupla: a esteira passa 2 latas/s; o sistema dá conta?	Checkpoint: tabela de latência · Quiz relâmpago 2
115–145	Desafio da dupla	Com 20 imagens de teste: (1) testar limiares de confiança 0,25 / 0,5 / 0,8; (2) contar quantos defeitos passaram e quantas latas boas foram descartadas; (3) preencher a ficha de decisão: limiar escolhido + "aprovo ou não para a linha" com justificativa.	PBL + pares · professor circula
145–160	Dúvidas e erros comuns	Cada dupla traz o erro que encontrou. O professor resolve no projetor 2–3 casos (tabela da seção 6) mostrando como diagnosticar.	Erro como conteúdo
160–175	Síntese	Retomar os 6 objetivos e marcar juntos quais foram atingidos. 3 duplas apresentam sua decisão em 1 minuto cada. Fechamento em uma frase: "Edge AI é levar o modelo até a máquina, do tamanho que a máquina aguenta."	Verificação coletiva
175–180	Próximos passos	Ticket de saída (2 perguntas) e apresentação da próxima aula.	Avaliação formativa

5. Instrumentos de avaliação

Avaliação formativa (durante a aula)

- **Quiz relâmpago 1 e 2** (3 questões cada, votação com cartões ou formulário): mostram ao professor se pode avançar ou se precisa reexplicar.
- **Checkpoints dos degraus:** a dupla mostra a tela; o professor registra quem precisou de ajuda e em qual etapa.
- **Ticket de saída:** "Qual frase define Edge AI?" e "Que número do JSON você olharia primeiro para confiar numa detecção?"

Rubrica da entrega da dupla (ficha de decisão + respostas da API)

Critério (peso)	Insuficiente (0)	Básico (1)	Proficiente (2)	Avançado (3)
Execução (20%)	Container não subiu	Subiu com ajuda do professor	Subiu e respondeu ao <code>/saude</code> sozinha	Subiu e explicou porta e <code>restart</code>
Uso da API (20%)	Nenhuma resposta válida	Só um endpoint	JSON e PNG obtidos	Testou também um erro (arquivo inválido)
Leitura dos resultados (20%)	Não interpreta o JSON	Identifica classes	Relaciona confiança com acerto	Aponta falsos positivos e negativos
Análise de desempenho (20%)	Sem medidas	Mediu só um formato	Comparou PyTorch × NCNN	Relacionou ms com latas/s da esteira
Decisão técnica (20%)	Sem decisão	Decisão sem justificativa	Limiar justificado com dados	Considerou o custo de cada tipo de erro

Nota final = soma ponderada (0 a 3) convertida para 0–10. A rubrica é entregue no início da aula: os estudantes sabem como serão avaliados.

6. Gestão de dificuldades (erros previstos)

Erro comum	Causa provável	Como o professor conduz
Build do container demora ou falha	Rede lenta, falta de espaço, arquitetura errada	Imagem já montada antes da aula; mostrar <code>docker images</code> e <code>docker buildx</code> como diagnóstico
Porta 8000 ocupada	Outro serviço ou container antigo	Ensinar <code>docker ps</code> e trocar a porta no <code>compose</code>
"Modelo não carrega"	Pasta do modelo ausente ou caminho errado	Ler o log com <code>docker compose logs</code> ; conferir <code>/saude</code>
API responde 400	Arquivo não é imagem ou campo com nome errado	Ler a mensagem de erro do JSON: erro também é informação
Latência alta	Formato PyTorch na CPU, imagem grande	Vira o gancho do degrau 3: NCNN e tamanho de entrada
Deteção errada	Classe com poucos exemplos no treino (lata prateada lisa)	Discutir dados de treino: mais imagens daquele caso

Engajamento: placar das duplas por checkpoint no quadro; papel de navegador obrigatório (ninguém fica só olhando); desafio extra para quem termina antes; duplas que terminaram ajudam a dupla vizinha antes de receber o desafio extra.

7. Recursos e materiais de apoio (checklist antes da aula)

- ☐ Imagem `inspecao-latas` montada e testada em todas as máquinas (`/saude` respondendo)
- ☐ Repositório clonado e pasta com 20 imagens de teste em cada máquina
- ☐ Vídeo da esteira e slides no computador do professor; projetor testado
- ☐ Folha impressa: comandos (Docker, curl), rubrica e ficha de decisão da dupla
- ☐ Quizzes 1 e 2 prontos (cartões A/B/C/D ou formulário online)
- ☐ (Opcional) Kit Raspberry Pi 5 montado com LED no GPIO 17 simulando o soprador
- ☐ Plano B: uma máquina reserva com tudo pronto e a gravação da demonstração

8. Próximos passos na trilha

- **Próxima aula:** criar o próprio dataset (captura de vídeo, anotação e treino do YOLO).
- **Desafio opcional:** enviar um alerta MQTT a cada lata reprovada; ou exportar o modelo em INT8 e medir o ganho de latência e a perda de precisão.
- **Leitura complementar:** guia Ultralytics para Raspberry Pi e documentação de exportação de modelos.

Referências

ANDERSON, L. W.; KRATHWOHL, D. R. (org.). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman, 2001.

FREEMAN, S. et al. Active learning increases student performance in science, engineering, and mathematics. *PNAS*, v. 111, n. 23, p. 8410–8415, 2014.

MAZUR, E. *Peer Instruction: A User's Manual*. Upper Saddle River: Prentice Hall, 1997.

ULTRALYTICS. Quick Start Guide: Raspberry Pi with Ultralytics YOLO. Disponível em: docs.ultralytics.com/guides/raspberry-pi. Acesso em: 4 out. 2026.